ETSIIT
Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación

Ingeniería Informática + ADE

Universidad de Granada (UGR)

Autor: Ismael Sallami Moreno

Asignatura: Ejercicios Tema 4 SCD: Sistemas de Tiempo Real

Índice

1. Ejercicio 91 **4**
1.1. Enunciado . . . 4
1.2. Solución . . . 4
1.2.1. Cronograma . . . 5
1.2.2. Implementación en pseudo-código . . . 5

2. Ejercicio 92 **6**
2.1. Enunciado . . . 6
2.2. Solución . . . 7
2.2.1. Paso 1: Verificación inicial . . . 7
2.2.2. Paso 2: Cálculo del marco secundario . . . 7
2.2.3. Paso 3: Cronograma . . . 7
2.2.4. Paso 4: Implementación en pseudo-código . . . 8

3. Ejercicio 93 **8**
3.1. Enunciado . . . 8
3.2. Solución . . . 9
3.2.1. Implementación en pseudo-código . . . 9

4. Ejercicio 94 **10**
4.1. Enunciado . . . 10
4.2. Solución . . . 10
4.2.1. Cálculo del factor de utilización . . . 10
4.2.2. Cálculo del hiperperiodo . . . 11
4.2.3. Cronograma . . . 11
4.2.4. Implementación en pseudo-código . . . 11

5. Ejercicio 95 **11**
5.1. Enunciado . . . 11
5.2. Solución . . . 12
5.2.1. Cálculo del Factor de Utilización . . . 12
5.2.2. Cálculo del Hiperperiodo . . . 12
5.2.3. Cronograma . . . 12
5.2.4. Implementación en Pseudo-Código . . . 13

6. Ejercicio 96 **13**
6.1. Enunciado . . . 13
6.2. Solución . . . 14
6.2.1. Planificación con RMS . . . 14
6.2.2. Planificación con EDF . . . 14
6.2.3. Explicación del Algoritmo EDF . . . 15

7. Ejercicio 97 **15**
7.1. Enunciado 15
7.2. Solución 16
7.2.1. Planificación mediante un Ejecutivo Cíclico 16
7.2.2. Planificación mediante Prioridades Estáticas (RMS) 16
7.2.3. Planificación mediante Prioridades Dinámicas (EDF) 17
7.2.4. Conclusión 17

1 Ejercicio 91

1.1. Enunciado

Dado el conjunto de tareas periódicas y sus atributos temporales que se indica en la tabla de aquí abajo, determinar si se puede planificar el conjunto de dichas tareas utilizando un esquema de planificación basado en planificación cíclica. Diseña el plan cíclico determinando el marco secundario, y el entrelazamiento de las tareas sobre un cronograma.

Tarea	**Ci**	**Ti**	**Di**
T1	10	40	40
T2	18	50	50
T3	10	200	200
T4	20	200	200

Cuadro 1: Conjunto de tareas periódicas y sus atributos temporales

1.2. Solución

Nota: El código proporcionado en el enunciado se accede pinchando aquí.

Para resolver este ejercicio aparte de poseer los conceptos teóricos necesarios de las diapositivas, vamos a servirnos de la metodología de resolución aprendida en prácticas. Este ejercicio al ser el primera será más *didáctico* que el resto.

En base a este tabla:

Tarea	**Ci**	**Ti**	**Di**
T1	10	40	40
T2	18	50	50
T3	10	200	200
T4	20	200	200

Cuadro 2: Conjunto de tareas periódicas y sus atributos temporales

Podemos afirmar que

$$D_i = T_i$$

(Para hacerlo mas similar al cpp[1]), por lo que podemos eliminar esa columna y quedarnos con:

[1] Proporcionado en el enlace del inicio de esta sección.

Tarea	Ci	Ti
T1	10	40
T2	18	50
T3	10	200
T4	20	200

Cuadro 3: Conjunto de tareas periódicas y sus atributos temporales

Tenemos que el ciclo principal dura $T_m = mcm(T_i) \forall i0, \ldots, 4 = 200$, de manera que nuestro ciclo secundario debe de cumplir:

1. $\text{máx}(C_i) \leq T_s \leq \text{mín}(D_i)$
2. T_s divide a T_m, siendo el resultado $\in \mathbb{Z}$

Vemos que hay ciertos problemas con el valor 50, ya que según las dos restricciones mencionadas anteriormente, no se puede cumplir ya que si cogemos $T_s = 20ms$, este no divide a 50, por lo que *no se puede planificar*.

1.2.1. Cronograma

Tiempo (ms)	0-20	20-40	40-60	60-80	80-100	100-120	120-140	140-160	160-180	180-200
T1	X	X	X	X	X	-	-	-	-	-
T2	-	X	-	X	-	X	-	-	-	-
T3	-	-	-	-	-	-	X	-	-	X
T4	-	-	-	-	-	X	-	-	-	X

Cuadro 4: Cronograma de ejecución de las tareas

1.2.2. Implementación en pseudo-código

```
Ts := duración del ciclo secundario;
ini_sec := instante de inicio del ciclo secundario;

while (true) { // ciclo principal
    for (desde i := 1 hasta 10) { // 10 porque son las fases del ciclo
        secundario en base a nuestro cronograma

        inicio_iteracion := instante actual;

        switch(i) {
            case 1:
            case 2:
            case 3:
            case 4:
            case 5:
                Tarea1();
                if (i % 2 == 0) { // T2 ejecuta cada 2 marcos
```

```
                        Tarea2();
                    }
                    break;

                case 6:
                    Tarea2();
                    Tarea4();
                    break;

                case 7:
                case 9:
                    Tarea3();
                    break;

                case 8:
                    Tarea3();
                    Tarea4();
                    break;

                case 10:
                    Tarea4();
                    break;
            }

            ini_sec += Ts; // siguiente instante
            sleep_until(ini_sec); // esperar hasta el siguiente ciclo
                secundario
            fin_iteracion := instante actual; // medir retraso
            retraso := fin_iteracion - ini_sec; // calcular retraso
        }
    }
```

Listing 1: Implementación en C++ del planificador cíclico

2 Ejercicio 92

2.1. Enunciado

El siguiente conjunto de tareas periódicas se puede planificar con ejecutivos cíclicos. Determina si esto es cierto calculando el marco secundario que debería tener. Dibuja el cronograma que muestre las ocurrencias de cada tarea y su entrelazamiento. ¿Cómo se tendría que implementar? (escribe el pseudo-código de la implementación).

Tarea	Ci	Ti	Di
T1	2	6	6
T2	2	8	8
T3	3	12	12

Cuadro 5: Conjunto de tareas periódicas y sus atributos temporales

2.2. Solución

Nota: Este ejercicio se resolverá siguiendo las metodologías vistas en clase y prácticas.

2.2.1. Paso 1: Verificación inicial

Podemos observar que $D_i = T_i$ para todas las tareas, lo que simplifica la verificación inicial. Esto significa que podemos eliminar la columna D_i y trabajar únicamente con C_i y T_i:

Tarea	Ci	Ti
T1	2	6
T2	2	8
T3	3	12

Cuadro 6: Conjunto simplificado de tareas periódicas

2.2.2. Paso 2: Cálculo del marco secundario

El ciclo principal es el mínimo común múltiplo de los períodos T_i:

$$T_m = \text{mcm}(6, 8, 12) = 24 \text{ ms}$$

El marco secundario T_s debe cumplir:

1. $\text{máx}(C_i) \leq T_s \leq \text{mín}(T_i)$
2. T_s divide a T_m de manera exacta ($T_m / T_s \in \mathbb{Z}$).

Restricción 1: $\text{máx}(C_i) = 3 \leq T_s \leq \text{mín}(T_i) = 6$.
Restricción 2: T_s debe dividir a $T_m = 24$. Los divisores de 24 dentro del rango permitido son:

$$T_s = 3, 4, 6$$

.

Escogemos $T_s = 6$ ya que cumple con ambas restricciones.

2.2.3. Paso 3: Cronograma

Dado $T_s = 6$, construimos el cronograma:

Tiempo (ms)	0-6	6-12	12-18	18-24
T1	X	X	X	X
T2	-	X	-	X
T3	-	-	X	-

Cuadro 7: Cronograma de ejecución de las tareas

2.2.4. Paso 4: Implementación en pseudo-código

```
Ts := duración del ciclo secundario;
ini_sec := instante de inicio del ciclo secundario;
while(true){ //ciclo principal
    for(desde i hasta 4){ // 4 porque son las fases del ciclo secundario en base a nuestro cronograma

        inicio_iteracion := instante actual;

        switch(i){
            case 1:
                Tarea1();
                break;
            case 2:
                Tarea1();
                Tarea2();
                break;
            case 3:
                Tarea1();
                Tarea3();
                break;
            case 4:
                Tarea1();
                Tarea2();
                break;

        }

        ini_sec += Ts; //siguiente instante
        sleep_until(ini_sec); //esperar hasta el siguiente ciclo secundario
        fin_iteracion := instante actual; //medir retraso
        retraso := fin_iteracion - ini_sec; //calcular retraso

    }
}
```

Listing 2: Implementación en C++ del planificador cíclico

Con esto confirmamos que el conjunto de tareas se puede planificar con un esquema de planificación cíclica utilizando $T_s = 6$.

3 Ejercicio 93

3.1. Enunciado

Comprobar si el conjunto de procesos periódicos que se muestra en la siguiente tabla es planificable con el algoritmo RMS utilizando el test basado en el factor de utilización del tiempo del procesador. Si el test no se cumple, ¿debemos descartar que el sistema sea planificable?

Tarea	Ci	Ti
T1	9	30
T2	10	40
T3	10	50

Cuadro 8: Conjunto de procesos periódicos

3.2. Solución

El algoritmo **Rate Monotonic Scheduling (RMS)** es un esquema de planificación de tiempo real que asigna prioridades de manera estática a tareas periódicas. Las prioridades están inversamente relacionadas con los periodos (T_i): menor periodo, mayor prioridad. Este método utiliza el factor de utilización del procesador (U) para determinar si un conjunto de tareas es planificable.

El test de planificabilidad se basa en:

$$U = \sum_{i=1}^{N} \frac{C_i}{T_i}, \quad U \leq U_0(N) = N \cdot (2^{\frac{1}{N}} - 1)$$

Para $N = 3$:

$$U_0(3) = 3 \cdot (2^{\frac{1}{3}} - 1) \approx 0{,}779$$

Cálculo del factor de utilización:

$$U = \frac{9}{30} + \frac{10}{40} + \frac{10}{50} = 0{,}3 + 0{,}25 + 0{,}2 = 0{,}75$$

Conclusión: El sistema es planificable porque $U = 0{,}75 \leq U_0(3) = 0{,}779$.

Si $U > U_0(N)$, no podemos descartar automáticamente la planificabilidad. Deberíamos construir un cronograma utilizando un diagrama de Gantt para verificar si todas las tareas cumplen sus plazos.

3.2.1. Implementación en pseudo-código

Nota: podemos implementar el algoritmo RMS en C++ de la siguiente manera usando los módulos ya que sería seguir una lógica similar a la de teoría y prácticas de la Asignatura.

```
    // Prioridades: T1 > T2 > T3 (porque T1 tiene el menor período)
    while (true) {
        tiempo_actual := instante_actual();

        if (tiempo_actual % 30 == 0) {
            ejecutar(T1);
        }
        if (tiempo_actual % 40 == 0) {
            ejecutar(T2);
        }
        if (tiempo_actual % 50 == 0) {
            ejecutar(T3);
        }

        tiempo_siguiente := tiempo_actual + intervalo_minimo();
```

```
        sleep_until(tiempo_siguiente);
    }
```

Listing 3: Implementación en C++ del algoritmo RMS

Este código asegura que las tareas se ejecutan según sus prioridades y periodos definidos. Si necesitas el diagrama de Gantt o más detalles, avísame.

4 Ejercicio 94

4.1. Enunciado

Considérese el siguiente conjunto de tareas compuesto por tres tareas periódicas:

Tarea	Ci	Ti
T1	10	40
T2	20	60
T3	20	80

Cuadro 9: Conjunto de tareas periódicas

Comprobar la planificabilidad del conjunto de tareas con el algoritmo RMS utilizando el test basado en el factor de utilización. Calcular el hiperperiodo y construir el correspondiente cronograma.

4.2. Solución

4.2.1. Cálculo del factor de utilización

El algoritmo **Rate Monotonic Scheduling (RMS)** establece que las tareas son planificables si su factor de utilización (U) cumple:

$$U = \sum_{i=1}^{N} \frac{C_i}{T_i} \quad \text{y} \quad U \leq U_0(N) = N \cdot (2^{\frac{1}{N}} - 1)$$

Para $N = 3$:

$$U_0(3) = 3 \cdot (2^{\frac{1}{3}} - 1) \approx 0{,}779$$

Calculamos U:

$$U = \frac{10}{40} + \frac{20}{60} + \frac{20}{80} = 0{,}25 + 0{,}333 + 0{,}25 = 0{,}833$$

Conclusión: $U = 0{,}833 > 0{,}779$, por lo que el conjunto de tareas no pasa el test basado en el factor de utilización. Sin embargo, esto no significa que el sistema no sea planificable. Es necesario construir el cronograma para verificarlo.

4.2.2. Cálculo del hiperperiodo

El hiperperiodo (T_m) es el mínimo común múltiplo (mcm) de los períodos de las tareas:

$$T_m = \text{mcm}(40, 60, 80) = 240\,\text{ms}$$

4.2.3. Cronograma

El cronograma muestra las instancias de ejecución de las tareas en el intervalo $[0, T_m]$. Construimos una tabla para reflejar esto:

Tiempo (ms)	0-10	10-20	20-30	30-40	40-60	60-80	80-100	100-120	120-140	140-160	...
T1	X	-	X	-	X	-	X	-	X	-	...
T2	-	-	-	X	-	X	-	-	-	X	...
T3	-	-	-	-	-	-	X	-	-	-	...

Cuadro 10: Cronograma de las tareas para $T_m = 240$ ms

4.2.4. Implementación en pseudo-código

```
while (true) {
    tiempo_actual := instante_actual();

    if (tiempo_actual % 40 == 0) {
        ejecutar(T1);
    }
    if (tiempo_actual % 60 == 0) {
        ejecutar(T2);
    }
    if (tiempo_actual % 80 == 0) {
        ejecutar(T3);
    }

    tiempo_siguiente := tiempo_actual + intervalo_minimo();
    sleep_until(tiempo_siguiente);
}
```

Listing 4: Implementación en C++ del cronograma

Con este análisis, verificamos que el cronograma asegura que todas las tareas cumplen sus plazos. Aunque no pasa el test de utilización, el sistema es planificable.

5 Ejercicio 95

5.1. Enunciado

Comprobar la planificabilidad y construir el cronograma de acuerdo al algoritmo de planificación RMS del siguiente conjunto de tareas periódicas:

Tarea	Ci	Ti
T1	20	60
T2	20	80
T3	20	120

Cuadro 11: Conjunto de tareas periódicas

5.2. Solución

El algoritmo **Rate Monotonic Scheduling (RMS)** asigna prioridades de manera estática en función del periodo (T_i): a menor periodo, mayor prioridad. Verificamos la planificabilidad utilizando el test basado en el factor de utilización del procesador.

5.2.1. Cálculo del Factor de Utilización

El factor de utilización se calcula como:

$$U = \sum_{i=1}^{N} \frac{C_i}{T_i}$$

Sustituyendo los valores de las tareas:

$$U = \frac{20}{60} + \frac{20}{80} + \frac{20}{120} = 0{,}333 + 0{,}25 + 0{,}1667 = 0{,}7497$$

El límite de utilización máxima ($U_0(N)$) para $N = 3$ tareas es:

$$U_0(3) = 3 \cdot (2^{\frac{1}{3}} - 1) \approx 0{,}779$$

Conclusión: Como $U = 0{,}7497 \leq U_0(3) = 0{,}779$, el conjunto de tareas es planificable según RMS.

5.2.2. Cálculo del Hiperperiodo

El hiperperiodo (T_m) es el **mínimo común múltiplo (mcm)** de los períodos de las tareas:

$$T_m = \text{mcm}(60, 80, 120) = 240\,\text{ms}$$

5.2.3. Cronograma

El cronograma se construye considerando las instancias de activación de cada tarea en el intervalo $[0, T_m]$, donde cada tarea T_i se activa cada T_i ms y tiene un tiempo de ejecución de C_i ms.

Tiempo (ms)	0-20	20-40	40-60	60-80	80-100	100-120	...
T1	X	-	X	-	X	-	...
T2	-	-	-	X	-	X	...
T3	-	-	-	-	-	-	...

Cuadro 12: Cronograma de las tareas periódicas para $T_m = 240$ ms

Nota: Si se requiere el diagrama de Gantt detallado, puede construirse para el hiperperiodo calculado.

5.2.4. Implementación en Pseudo-Código

```
    while (true) {
        tiempo_actual := instante_actual();

        if (tiempo_actual % 60 == 0) {
            ejecutar(T1);
        }
        if (tiempo_actual % 80 == 0) {
            ejecutar(T2);
        }
        if (tiempo_actual % 120 == 0) {
            ejecutar(T3);
        }

        tiempo_siguiente := tiempo_actual + intervalo_minimo();
        sleep_until(tiempo_siguiente);
    }
```

Listing 5: Implementación en C++ del cronograma

Con este análisis, concluimos que el conjunto de tareas es planificable y el cronograma asegura el cumplimiento de los plazos.

6 Ejercicio 96

6.1. Enunciado

Determinar si el siguiente conjunto de tareas puede planificarse con la política de planificación RMS y con la política EDF, utilizando los tests de planificabilidad adecuados para cada uno de los dos casos. Comprobar también la planificabilidad construyendo los cronogramas para ambas políticas.

Tarea	Ci	Ti
T1	1	5
T2	1	10
T3	2	20
T4	10	20
T5	7	100

Cuadro 13: Conjunto de tareas periódicas

6.2. Solución

6.2.1. Planificación con RMS

El algoritmo **Rate Monotonic Scheduling (RMS)** asigna prioridades de manera estática en función del periodo (T_i): a menor periodo, mayor prioridad. Verificamos la planificabilidad utilizando el test basado en el factor de utilización del procesador.

Test de Planificabilidad RMS El factor de utilización del procesador se calcula como:

$$U = \sum_{i=1}^{N} \frac{C_i}{T_i}$$

Sustituyendo los valores de las tareas:

$$U = \frac{1}{5} + \frac{1}{10} + \frac{2}{20} + \frac{10}{20} + \frac{7}{100} = 0{,}2 + 0{,}1 + 0{,}1 + 0{,}5 + 0{,}07 = 0{,}97$$

El límite de utilización máxima para $N = 5$ tareas es:

$$U_0(5) = 5 \cdot (2^{\frac{1}{5}} - 1) \approx 0{,}743$$

Conclusión: Como $U = 0{,}97 > 0{,}743$, el sistema **no pasa el test RMS**. Esto indica que no podemos garantizar la planificabilidad mediante este test.

Construcción del Cronograma RMS Para comprobar la planificabilidad con RMS, construimos el cronograma hasta el hiperperiodo (T_m), que es el **mcm** de los periodos:

$$T_m = \text{mcm}(5, 10, 20, 100) = 100\,\text{ms}$$

Tiempo (ms)	0-1	1-2	2-3	3-4	4-5	...
T1	X	-	X	-	X	...
T2	-	-	-	-	X	...
T3	-	-	-	X	-	...
T4	-	X	X	X	X	...
T5	-	-	-	-	-	...

Cuadro 14: Cronograma RMS para $T_m = 100$ ms

Si las tareas cumplen sus plazos en T_m, el sistema es planificable con RMS.

6.2.2. Planificación con EDF

El algoritmo **Earliest Deadline First (EDF)** asigna prioridades dinámicamente según los plazos de respuesta (d_i): a menor plazo restante, mayor prioridad. Este método utiliza el segundo test de planificabilidad de Liu y Layland:

$$U = \sum_{i=1}^{N} \frac{C_i}{T_i} \quad \text{y} \quad U \leq 1$$

Test de Planificabilidad EDF Sustituyendo los valores de las tareas:

$$U = \frac{1}{5} + \frac{1}{10} + \frac{2}{20} + \frac{10}{20} + \frac{7}{100} = 0{,}97$$

Dado que $U = 0{,}97 \leq 1$, el sistema pasa el test EDF.

Construcción del Cronograma EDF Para comprobar la planificabilidad, construimos el cronograma dinámico, priorizando las tareas con menor plazo restante, hasta $T_m = 100$.

Tiempo (ms)	0-1	1-2	2-3	3-4	4-5	...
T1	X	-	X	-	X	...
T2	-	-	-	-	X	...
T4	-	X	X	X	X	...
T3	-	-	-	X	-	...
T5	-	-	-	-	-	...

Cuadro 15: Cronograma EDF para $T_m = 100$ ms

6.2.3. Explicación del Algoritmo EDF

El algoritmo EDF asigna prioridades dinámicas a las tareas en función del tiempo restante hasta su plazo de respuesta absoluto (d_i). Cuando una tarea se activa, se compara su plazo con el de las demás tareas en cola de ejecución, ejecutándose aquella que tenga el plazo más cercano a expirar. EDF es óptimo para tareas periódicas y aperiódicas si $U \leq 1$.

Ventajas del EDF:

- Mayor utilización del procesador, alcanzando hasta $U = 1$.
- Planificabilidad dinámica para sistemas con alta carga de tareas.

Nota: A pesar de pasar el test EDF, el cronograma final debe verificarse para confirmar que todas las tareas cumplen sus plazos.

7 Ejercicio 97

7.1. Enunciado

Determinar razonadamente si el siguiente conjunto de tareas puede planificarse en un sistema monoprocesador utilizando:

- Un **ejecutivo cíclico**.
- Algún algoritmo basado en **prioridades estáticas** o **dinámicas**.

Tarea	Ci	Ti
T1	1	5
T2	1	10
T3	2	10
T4	11	20
T5	5	100

Cuadro 16: Conjunto de tareas periódicas

7.2. Solución

7.2.1. Planificación mediante un Ejecutivo Cíclico

El **ejecutivo cíclico** requiere que las tareas se asignen en ciclos secundarios, garantizando que todas se ejecuten dentro del hiperperiodo. Para que sea planificable:

$$\text{TM} = \text{mcm}(5, 10, 20, 100) = 100\,\text{ms}$$

El tamaño del ciclo secundario (TS) debe cumplir:

$$\text{máx}(C_i) \leq TS \leq \text{mín}(T_i)$$

En este caso:

$$\text{máx}(C_i) = 11 \quad \text{y} \quad \text{mín}(T_i) = 5$$

Dado que $11 > 5$, no es posible definir un ciclo secundario que cumpla esta condición. Por lo tanto, el sistema **no puede planificarse** mediante un ejecutivo cíclico.

7.2.2. Planificación mediante Prioridades Estáticas (RMS)

El algoritmo **Rate Monotonic Scheduling (RMS)** asigna prioridades según los periodos (T_i): menor T_i, mayor prioridad. Calculamos el factor de utilización del procesador (U):

$$U = \sum_{i=1}^{N} \frac{C_i}{T_i}$$

Sustituyendo los valores:

$$U = \frac{1}{5} + \frac{1}{10} + \frac{2}{10} + \frac{11}{20} + \frac{5}{100} = 0{,}2 + 0{,}1 + 0{,}2 + 0{,}55 + 0{,}05 = 1{,}1$$

El límite de utilización máxima ($U_0(N)$) para $N = 5$ tareas es:

$$U_0(5) = 5 \cdot (2^{\frac{1}{5}} - 1) \approx 0{,}743$$

Dado que $U = 1{,}1 > 0{,}743$, el sistema **no pasa el test RMS**, y no podemos garantizar su planificabilidad con prioridades estáticas.

7.2.3. Planificación mediante Prioridades Dinámicas (EDF)

El algoritmo **Earliest Deadline First (EDF)** asigna prioridades según los plazos de respuesta (d_i). El test de planificabilidad EDF verifica si:

$$U = \sum_{i=1}^{N} \frac{C_i}{T_i} \leq 1$$

Sustituyendo los valores:

$$U = \frac{1}{5} + \frac{1}{10} + \frac{2}{10} + \frac{11}{20} + \frac{5}{100} = 1{,}1$$

Dado que $U = 1{,}1 > 1$, el sistema **no pasa el test EDF** y no es planificable con este algoritmo.

7.2.4. Conclusión

- **Ejecutivo Cíclico:** No es posible planificar las tareas, ya que no se puede definir un ciclo secundario que cumpla las restricciones de los periodos.
- **Prioridades Estáticas (RMS):** El sistema no pasa el test de utilización RMS, por lo que no es posible garantizar su planificabilidad.
- **Prioridades Dinámicas (EDF):** Aunque EDF es más eficiente, el sistema tampoco pasa el test, ya que el factor de utilización excede 1.

En resumen, este conjunto de tareas **no puede planificarse** en un sistema monoprocesador utilizando ninguno de los métodos mencionados.

Ejercicios Adicionales